2. 재귀와 분할 정복 CSED331 · 알고리즘 전체 홈↗

이 페이지

1. Reduction과 재귀
2. Tower of Hanoi
3. Binary search
4. 보충 · 구간 최댓값을 분할 정복으로
5. Mergesort
6. Quicksort
7. 점화식과 master theorem
8. 정수 곱셈과 Karatsuba
9. 보충 · Strassen 행렬 곱셈
10. 선형 시간 선택 (selection)
11. Closest pair of points
12. 연습문제

재귀 (recursion)와 분할 정복

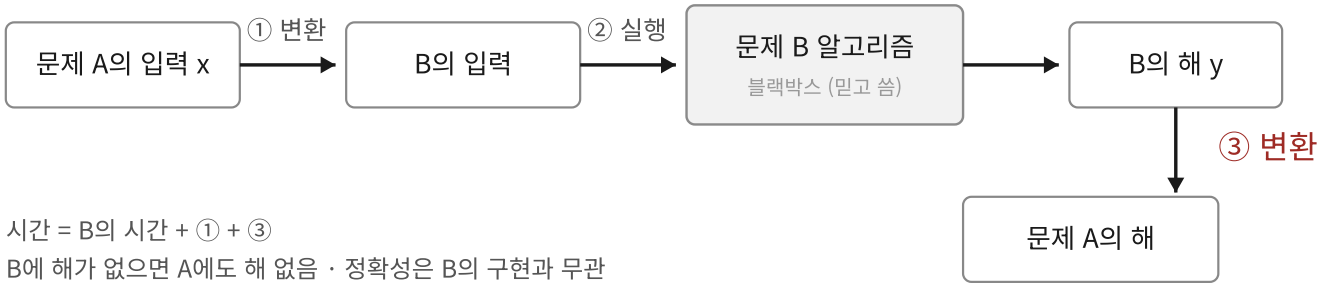
2026-09-09 (수) · 09-14 (월) · 슬라이드 Recursion (p.1–27, 녹취 없음, 교수 marked PDF)

Reduction과 재귀

> **reduction = 문제 A를 풀 때 문제 B를 푸는 알고리즘을 부품(블랙박스)으로 쓰는 것. B는 빠르게 푼다고 가정하고 그걸 가져다 A를 품.**

3단계	① A의 입력을 B의 입력으로 변환 → ② 블랙박스 실행 → ③ B의 해를 A의 해로 변환
시간	전체 시간은 입력 변환 비용, 모든 블랙박스 호출 비용의 합, 출력 변환 비용을 더한 것이다. B를 한 번 호출하면 기존 식으로 단순화되며, 여러 번 호출하면 각 호출의 입력 크기에 따른 실행 시간을 모두 합산한다. 변환이 싸면 전체 시간은 B가 좌우
정확성	Reduction의 두 변환이 올바르게 정의되어 A의 입력과 B의 인스턴스, B의 해와 A의 해 사이의 대응을 보존한다고 가정한다. 이 조건에서 B를 정확하게 푸는 블랙박스를 사용하면 A도 정확하게 풀 수 있다. B에 해가 없으면 A에도 해 없음

reduction: B를 블랙박스로 써서 A를 풂



재귀 = A와 B가 같은 문제인 reduction (B의 인스턴스가 더 작음)

재귀 = 같은 문제로의 reduction

- A와 B가 **같은 문제**인 reduction. 단, B의 인스턴스는 A보다 작아야 함 (예: 100개 정렬 → 50개 정렬 블랙박스 2번 호출)
- 직접 풀 수 있으면 직접 풂 (**base case**), 아니면 더 작은 인스턴스로 바꿈

설계할 때 정할 3가지

- 더 작은 인스턴스를 **어떻게** 만들지
- **base case**가 무엇인지
- 작은 문제의 해에서 **원래 해를 어떻게** 얻을지

주의: 작아지는 과정이 반드시 base case에 닿아야 함. 정확성 증명 = **귀납법** (base case + 작은 인스턴스가 맞다는 가정).

> 예: factorial. f(1)=1 → f(2)=2 → f(3)=6 → f(4)=24 → f(5)=120.

Peasant multiplication

$$x \cdot y = \begin{cases} (x/2) \cdot (2y) & x \text{ 짝 수} \\ ((x-1)/2) \cdot (2y) + y & x \text{ 홀 수} \end{cases}$$

- 반복하면 x가 0 (base case)에 도달. 가정: a는 0 이상의 정수, b는 정수. 표현 범위를 넘는 큰 수의 곱셈에 쓰려면 임의 정밀도 연산이 받쳐 줘야 하며, 이 알고리즘 자체가 오버플로를 해결하지는 않는다. 효율이 아니라 **재귀의 정의**를 보이는 예

손계산 1 · 11 × 13

단계	$x \cdot y$	변형	값 (되돌아오며 확정)
1	11·13	5·26 + 13	143
2	5·26	2·52 + 26	130
3	2·52	1·104	104
4	1·104	0·208 + 104	104
5	0·208	base case	0

- 끝까지 내려간 뒤 **아래에서 위로** 값이 확정됨
- 홀수 x마다 y를 더함 → 13 + 26 + 104 = 143. 11의 이진수 1011에서 1인 자리 → 13·1 + 13·2 + 13·8과 같음

Tower of Hanoi

기둥 A, B, C 중 A의 n개 원판을 C로 옮기는 문제. 한 번에 한 개만 이동, 큰 원판을 작은 원판 위에 놓을 수 없음, 남은 기둥은 임시 자리(spare)로 사용.

- 작은 경우부터 보면 규칙이 보임: $n = 1$이면 A → C 한 번 (base case). $n = 2$면 작은 것 → B, 큰 것 → C, 작은 것 B → C. $n = 3$이면 위 2개 옮기기를 아는 블랙박스에 두 번 맡김

하노이 탑 · 재귀 호출 추적

원판 n개를 A에서 C로 옮긴다. 한 번에 한 원판만 옮기며, 큰 원판은 작은 원판 위에 놓을 수 없다.

원판 수 n 3개 최소 7회



이동 0 / $2^3-1 = 7$

시작 상태

이전 수 | 다음 수 | 자동 재생 | 처음으로

시작: 3개 원판을 A에서 C로 옮깁니다. 가장 작은 부분문제부터 이동이 발생합니다.

현재 재귀 호출 스택

가장 바깥 호출부터 현재 이동이 발생한 호출까지

`hanoi(3, A→C) · 시작`

아직 이동하지 않았습니다. 다음 수를 눌러 호출을 추적하세요.

재귀 규칙

① n−1개를 보조 기둥으로 이동

② 가장 큰 원판 1개 이동

③ n−1개를 목적 기둥으로 이동

$H(3) = 2 \cdot H(2) + 1$

H(1)=1. 각 단계에서 n−1개를 옮기는 호출 2회와 가장 큰 원판의 이동 1회가 필요합니다.

함정	블랙박스를 두 번 쓰지만 **목적지가 다름**. 첫 호출의 도착은 B, 둘째 호출의 출발은 B
인자 순서	**Hanoi(n, 출발, 도착, spare)**가 헷갈리는 부분. Hanoi(n−1, A, B, C) → 이동 → Hanoi(n−1, B, C, A)

이동 횟수

- 호출이 2번 → 크기가 1 줄 때마다 횟수 2배

$$T(n) = 2T(n-1) + 1 = 2^n - 1$$

- $T(n) = 2T(n-1) + 1$ 꼴이지 $T(n/b)$ 꼴이 아님 → **master theorem 형태가 아님** (크기가 나뉘지 않고 1 줄어듦) → 펼치기로 풂

손계산 2 · 이동 횟수

n	이동 횟수	recurrence	$2^n - 1$
0	0	base case	0
1	1	2 × 0 + 1	1
2	3	2 × 1 + 1	3
3	7	2 × 3 + 1	7
4	15	2 × 7 + 1	15
5	31	2 × 15 + 1	31
6	63	2 × 31 + 1	63

- 원판 하나 추가 → 횟수 2배 + 1. 64개: $2^{64} - 1 = 18446744073709551615$번 → 초당 1번이어도 약 10^{19}초 규모

손계산 3 · $n = 3$ 이동 순서

원판 번호: 1번이 가장 작고 n번이 가장 큼 (Hanoi(3, A, C, B))

이동 순서

이동 1: 1번 A → C · 이동 2: 2번 A → B · 이동 3: 1번 C → B · 이동 4: **3번 A → C** · 이동 5: 1번 B → A · 이동 6: 2번 B → C · 이동 7: 1번 A → C

- 가장 큰 원판은 가운데(2^{n-1}번째) 이동에서 **한 번만** 움직임
- 원판 k번의 이동 횟수 = 2^{n-k} (1번 4회, 2번 2회, 3번 1회 → 합 7)

Binary search

sorted list L에서 key v 찾기. 구간을 반씩 버리기:

- base case: 구간이 비어 있으면, 즉 lo > hi이면 탐색 실패를 반환한다. 구간에 원소가 하나만 있으면 그 원소와 key를 비교해 성공 또는 실패를 반환한다
- 그 외에는 가운데 원소와 비교해 한쪽 구간만 탐색한다
- 비교 한 번에 문제 크기가 절반. **재귀 가지 1개** (mergesort는 2개)
- 확정되는 것: "왼쪽에 없다" (오른쪽에 있다는 게 아님) → key가 아예 없을 수도 있음. worst case: 없는 key이거나 마지막에 찾음

> 크기가 1이 되기까지 $\log_2 n$ 단계, 최악 비교 $\lfloor \log_2 n \rfloor + 1$

- 예: 5천만 건 기록에서 ID로 찾기 → 최대 26번 비교 ($\log_2(5 \times 10^7) \approx 25.6$)

슬라이드 표기: m은 양 끝 포함, 구간 안의 상대 위치 → 실제 인덱스는 lo + m. 아래 손계산은 0-based 인덱스 사용.

손계산 4 · $L = [1, 2, 3, 5, 6, 7, 10, 13]$ 탐색

mergesort 손계산의 정렬 결과, 인덱스 0-7. key = 7 (있음):

- 단계 1: lo 0, hi 7, mid 3, L[mid] 5, 비교 L[mid] < key → lo = 4
- 단계 2: lo 4, hi 7, mid 5, L[mid] 7, 비교 같음 → 찾음

key = 4 (없음):

단계	lo	hi	mid	L[mid]	비교	다음
1	0	7	3	5	L[mid] > key	hi = 2

2	0	2	1	2	L[mid] < key	lo = 2
3	2	2	2	3	L[mid] < key	lo = 3
4	3	2	-	-	lo > hi	없음

- 구간이 8 → 3 → 1 → 0으로 줄어듦. 최악 비교 횟수 $\lfloor \log_2 n \rfloor + 1$: 8개 4번, 15개 4번, 16개 5번

보충 · 구간 최댓값을 분할 정복으로

보충: 이 절은 강의 슬라이드에 직접 등장하지 않는 추가 예제다. 분할 정복의 기본 틀(나누기 → 각 부분을 재귀로 풀기 → 결합)을 가장 단순한 문제에 적용해 본다.

문제: 배열 A의 구간 [i, j]에서 가장 큰 원소, 즉 구간 최댓값을 구한다. 구간을 반으로 나눠 양쪽의 최댓값을 각각 구하면, 전체 최댓값은 둘 중 더 큰 값이다.

- 기저: i = j이면 원소가 하나뿐이라 그 값 A[i]가 곧 최댓값
- 분할: mid = ⌊(i + j) / 2⌋로 나눈 왼쪽 구간 [i, mid]와 오른쪽 구간 [mid + 1, j]
- 결합: 왼쪽 최댓값과 오른쪽 최댓값 중 더 큰 값. 비교 1회면 충분

```
Max(A, i, j)
  if i = j
    return A[i]
  mid ← ⌊(i + j) / 2⌋
  left ← Max(A, i, mid)
  right ← Max(A, mid + 1, j)
  return max(left, right)
```

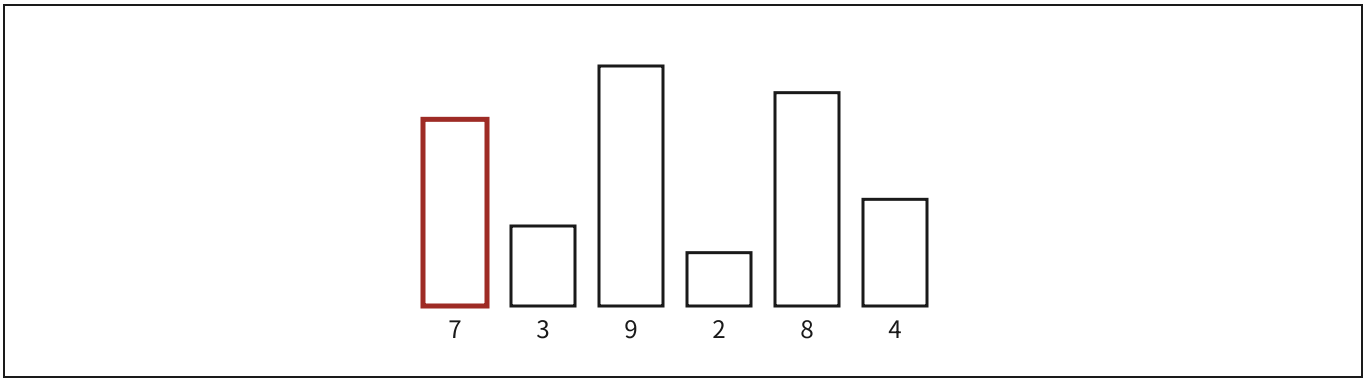
예제 추적 · A = [4, 7, 2, 9, 1, 6, 3, 8], 구간 [0, 7]

- 구간 [0, 7], mid = 3 → 왼쪽 [0, 3], 오른쪽 [4, 7]
- 구간 [0, 3], mid = 1 → [0, 1]은 max(4, 7) = 7, [2, 3]은 max(2, 9) = 9 → 결합 max(7, 9) = 9
- 구간 [4, 7], mid = 5 → [4, 5]는 max(1, 6) = 6, [6, 7]은 max(3, 8) = 8 → 결합 max(6, 8) = 8
- 최종 결합: max(9, 8) = **9**

복잡도

- 점화식: $T(n) = 2T(n/2) + \Theta(1)$. 부분문제 2개, 결합은 비교 1회뿐
- master theorem을 적용하면 $T(n) = \Theta(n)$
- 비교 횟수는 정확히 $n - 1$회. 처음부터 끝까지 한 번 훑는 순차 탐색의 비교 횟수와 완전히 같다
- 즉 분할 정복을 쓴다고 항상 이득이 생기는 것은 아니다. 이 문제에서는 비교 횟수 기준으로 순차 탐색과 동률이다

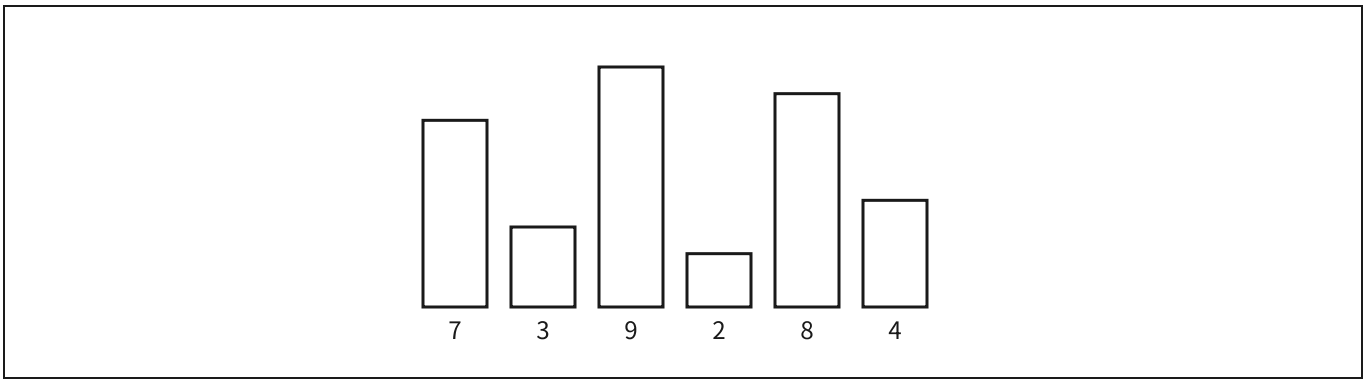
삽입 정렬 (Insertion Sort)



초기 배열. 앞부분(인덱스 0)은 이미 정렬된 것으로 본다.

◀ 이전 1 / 20 다음 ▶

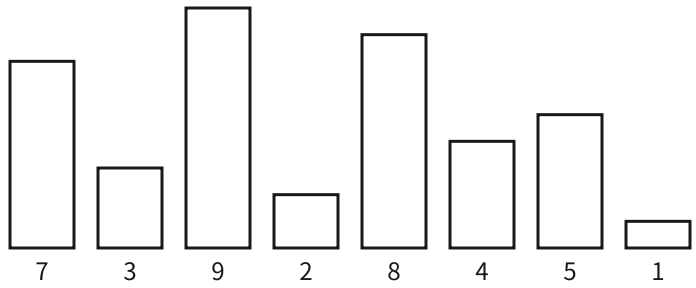
선택 정렬 (Selection Sort) — 보충



보충: 삽입 정렬과 비교하는 선택 정렬 시각화. 강의 슬라이드 직접 근거 미확인. 초기 배열. 미정렬 구간의 최솟값을 찾아 앞 원소와 교환하는 선택 정렬(Selection Sort)을 시작한다.

◀ 이전 1 / 27 다음 ▶

병합 정렬 (Merge Sort)



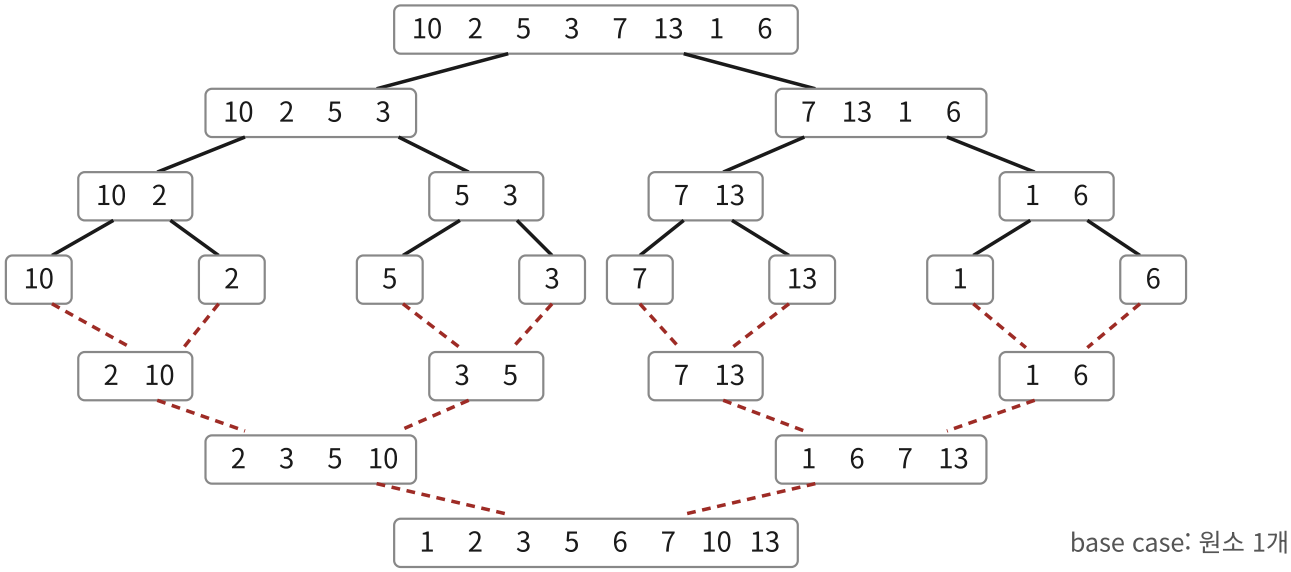
초기 배열. 병합 정렬은 반으로 나누고(분할) 각각을 정렬한 뒤 합친다(병합).

◀ 이전 1 / 16 다음 ▶

Mergesort

- **3단계** (base case $n \leq 1$): ① 리스트를 반으로 나눔 → ② 양쪽을 재귀로 mergesort (simpler instance 2개) → ③ 두 sorted list를 하나로 **merge**
- 1945년 von Neumann
- 맨 위 단계는 나누고 합치기만 함, 나머지 일은 아래 단계에 맡김
- merge: 두 list의 맨 앞 원소(포인터 2개)를 비교해 작은 쪽을 꺼내고 그 포인터만 전진. 한쪽이 비면 나머지를 그대로 붙임. 원소 n개 병합 = 비교 최대 $n - 1$번, 선형 시간

mergesort: [10 2 5 3 7 13 1 6] — 아래로 분할, 위로 병합



레벨마다 원소 8개를 병합 × 레벨 log n 개 → Θ(n log n)

> 모든 레벨에서 원소 n개를 병합, 레벨 수 $\log_2 n$ $\Rightarrow$ $\Theta(n \log n)$

- **입력 값과 무관** → 최선·최악 모두 $\Theta(n \log n)$ (quicksort와 대비)
- 비교 기반 정렬의 하한도 $\Omega(n \log n)$ → 이론상 최적. 비교 없이 순위로 넣는 특수한 경우(카드 번호 정렬 등)는 예외

정확성 (귀납법)

merge	합친 원소 수로 거는 귀납. 두 sublist가 sorted면 결과도 sorted
mergesort	n으로 거는 귀납. $n \leq 1$이면 할 일 없음. 그 외: 귀납 가정으로 크기 $< n$인 두 sublist가 맞게 정렬되고 merge가 둘을 하나의 sorted list로 합침

손계산 5 · $n = 8$

분할: [10 2 5 3] [7 13 1 6] → [10 2] [5 3] [7 13] [1 6] → 원소 1개짜리 8개 (base case). 맨 위 병합: [2 3 5 10] + [1 6 7 13], 슬라이드 표기 포인터 i (왼쪽), j (오른쪽):

k	i	j	비교	꺼낸 값

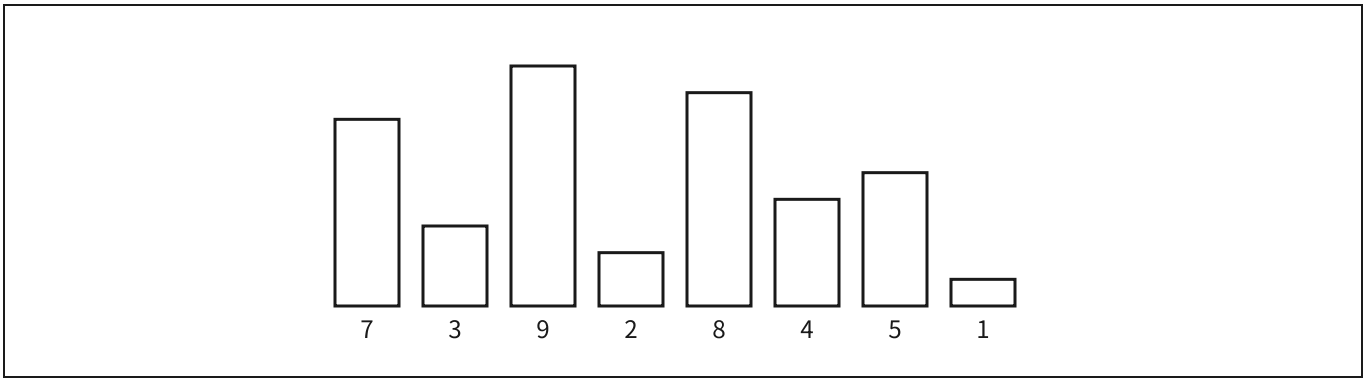
1	1	5	2 > 1	1
2	1	6	2 < 6	2
3	2	6	3 < 6	3
4	3	6	5 < 6	5
5	4	6	10 > 6	6
6	4	7	10 > 7	7
7	4	8	10 < 13	10
8	5	8	-	13

- 비교 7번 = $n - 1$. 마지막 13은 왼쪽이 비어 비교 없이 붙임

레벨	병합 횟수	병합당 원소 수	원소 합	비교 횟수
병합 3 (맨 아래)	4	2	8	4
병합 2	2	4	8	5
병합 1 (맨 위)	1	8	8	7
합			**24**	**16**

- 레벨마다 원소 8개를 병합 → 24 = $n \log_2 n$. 비교는 레벨마다 최대 $n - 1$ (병합 횟수만큼 뺌) → 이 입력은 4 + 5 + 7 = 16번. 최악은 4 + 6 + 7 = 17번 (모든 병합이 $n - 1$번 비교)인데 아래쪽 두 번째 병합 [7, 13] + [1, 6]이 2번으로 끝나 1번 적음

퀵 정렬 (Quick Sort)



초기 배열. 마지막 원소를 피벗으로 잡고 작은 값을 앞으로 모은다 (Lomuto 분할).

◀ 이전 1 / 24 다음 ▶

Quicksort

- 1959년 Tony Hoare. base case $n \leq 1$
- **3단계**: ① pivot 하나를 고름 → ② **partition**: pivot보다 작은 것 / pivot / 큰 것 → ③ 작은 쪽과 큰 쪽만 재귀 (simpler instance 2개)
- mergesort와 반대: **나눌 때 일하고 합칠 땐 할 일 없음**. 작은 쪽을 정렬해도 pivot 왼쪽 구역 안에서만 움직임

partition 코드 요점

- pivot을 맨 뒤로 보냄. ℓ = pivot보다 작은 원소 수
- $A[i] < pivot$이면 ℓ 증가 후 $A[i]$와 $A[\ell]$ 교환
- 마지막에 pivot을 $\ell + 1$로 → ℓ 반환. 단, ℓ은 pivot의 인덱스가 아니라 작은 원소의 개수이고, pivot의 최종 인덱스는 ℓ+1이다. 호출부는 왼쪽 ℓ개 구간과 ℓ+2번째부터의 구간을 재귀한다. 같은 값은 오른쪽 구역에 남음 (코드가 <만 비교)
- $O(n)$ 시간, 정확성은 partition이 맞다는 가정 위의 귀납

손계산 6 · partition 추적

입력 [10, 2, 5, 3, 7, 13, 1, 6], pivot = A[4] = 3 (가운데 위치 4번째):

i	A[i]	A[i] < 3	ℓ	배열
swap A[4] ↔ A[8] (pivot을 뒤로)				[10, 2, 5, 6, 7, 13, 1, 3]
1	10	아니오	0	[10, 2, 5, 6, 7, 13, 1, 3]
2	2	예	1	[2, 10, 5, 6, 7, 13, 1, 3]
3	5	아니오	1	[2, 10, 5, 6, 7, 13, 1, 3]

4	6	아니오	1	[2, 10, 5, 6, 7, 13, 1, 3]
5	7	아니오	1	[2, 10, 5, 6, 7, 13, 1, 3]
6	13	아니오	1	[2, 10, 5, 6, 7, 13, 1, 3]
7	1	예	2	[2, 1, 5, 6, 7, 13, 10, 3]
swap A[8] ↔ A[ℓ+1]			2	[2, 1, **3**, 6, 7, 13, 10, 5]

- $\ell = 2$: 왼쪽 [2, 1], pivot 3, 오른쪽 [6, 7, 13, 10, 5]

재귀 전체 (매번 가운데 위치를 pivot으로):

호출	구간	pivot	왼쪽	오른쪽	비교
1	[10, 2, 5, 3, 7, 13, 1, 6]	A[4] = 3	[2, 1]	[6, 7, 13, 10, 5]	7
2	[2, 1]	A[1] = 2	[1]	[]	1
3	[6, 7, 13, 10, 5]	A[3] = 13	[6, 7, 5, 10]	[]	4
4	[6, 7, 5, 10]	A[2] = 7	[6, 5]	[10]	3
5	[6, 5]	A[1] = 6	[5]	[]	1
합					**16**

- 호출 3은 pivot 13이 최댓값 → 오른쪽이 빔 (불균형). partition 한 번의 비교 = 구간 크기 − 1

최선·최악·평균

두 조각의 크기는 pivot의 순위 r에 달림

최선	항상 중앙값 → $\Theta(n \log n)$
최악	$r = 1$ 또는 $r = n$ → 이미 정렬된 입력 + 끝 원소 pivot → 비교 $n(n-1)/2$번. **3개 중 중앙값을 써도 최악은 $\Theta(n^2)$**
평균	보통 순위가 $n/4$와 $3n/4$ 사이 → $\Theta(n \log n)$

n	**최악** $n(n-1)/2$	$n \log_2 n$
8	28	24
16	120	64
1024	523776	10240

- pivot이 항상 가운데 1/3 안이면 레벨의 크기 합이 $(2/3 + 1/3)n = n$ 이하, 깊이 $O(\log n)$ → $O(n \log n)$. 짧은 가지가 먼저 끝나면 그 뒤 레벨 합은 n보다 작아짐
- 나쁜 pivot은 그 레벨에서만 나쁨 → 다음 레벨에서 pivot을 다시 고름. 나쁜 선택이 연속될 확률은 빠르게 작아지지만 0은 아니다. 무작위 pivot의 기대 시간은 Θ(n log n)이지만 최악 시간은 여전히 Θ(n^2)이고, 무작위화는 최악을 제거하지 않고 나쁜 분할이 계속될 가능성을 낮출 뿐이다
- pivot 고르는 요령: 무작위 1개 / 3개 중 중앙값 / 5개 중 셋째. 뽑고 비교하는 시간이 추가로 듦 → 3개 중앙값 정도로 대체로 충분. 중앙값은 $O(n)$에 구할 수 있음 (뒤의 selection), 단 상수가 큼

mergesort vs quicksort

항목	**mergesort**	**quicksort**
분할	항상 반반	pivot 순위에 따라 다름
합치기	merge $O(n)$	없음 (partition이 $O(n)$)
입력 의존	없음	있음
최악	$\Theta(n \log n)$	$\Theta(n^2)$
평균	$\Theta(n \log n)$	$\Theta(n \log n)$
추가 공간	병합용 배열 $O(n)$ + 재귀 스택 O(log n)	제자리 교환 (partition은 추가 배열 없음), 재귀 스택 평균 O(log n)·최악 O(n)
실전	이론상 최적	정렬 라이브러리가 변형을 많이 씀

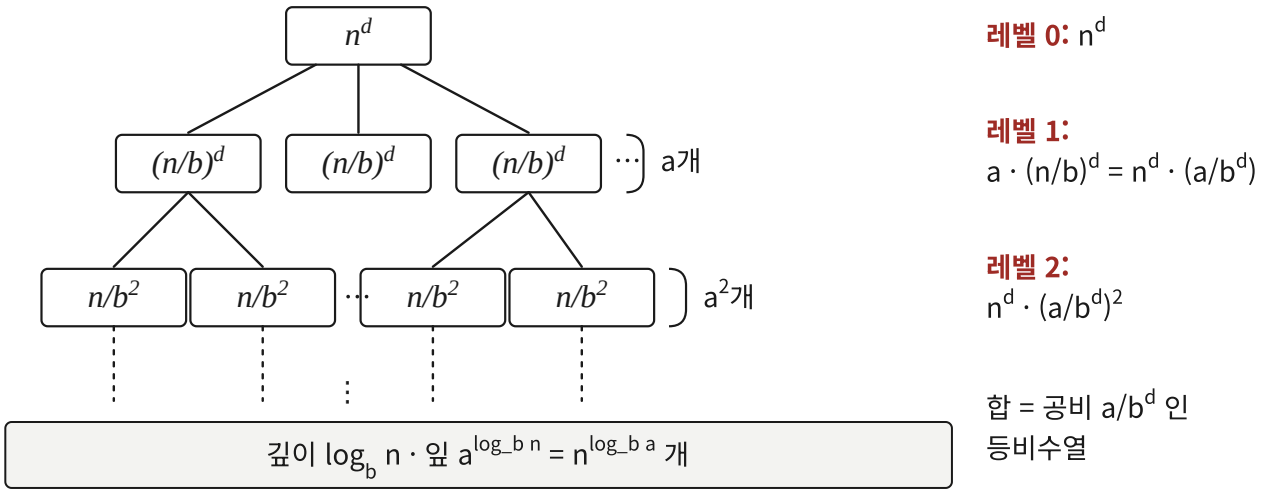
점화식과 master theorem

분할정복 점화식을 읽는 도구. 세 글자만 읽으면 됨:

a	재귀 호출 수 = 가지 수(branching factor)
b	입력이 줄어드는 비율 (부분문제 크기 n/b)
n^d	나누고 합치는 나머지 비용 (mergesort의 merge, quicksort의 partition). binary search는 비교 1번 → $d = 0$

$$T(n) = a \cdot T(n/b) + \Theta(n^d) \quad \Rightarrow \quad \log_b a \text{ 와 } d \text{ 를 비교}$$

재귀 트리



- k번째 레벨: 부분문제 a^k개, 크기 n/b^k. 깊이 $\log_b n$, 맨 아래 크기 1
- 잎의 수: $a^{\log_b n} = n^{\log_b a}$
- 레벨 k의 비용: $a^k(n/b^k)^d = n^d(a/b^d)^k$ → 전체 = 공비 a/b^d인 등비수열의 합

전제: 점화식은 T(n) = aT(n/b) + Θ(n^d) 꼴로 통일한다.
a, b는 양의 상수이고 b > 1이며, 부분문제 크기가 모두 n/b로 같은 비율로 줄어야 한다.
비재귀 비용이 일반적인 f(n)이면 확장된 Master theorem의 적용 조건을 확인해야 한다.
조건을 만족하지 않거나 적용 여부가 불분명하면 재귀 트리, 점화식 펼치기 또는 치환법으로 분석한다.

Master theorem의 케이스 3개

케이스	공비 a/b^d	레벨 비용	합을 정하는 것	결과
decreasing	< 1 ($d > \log_b a$)	줄어듦	첫 항(루트) n^d	$\Theta(n^d)$
equal	$= 1$ ($d = \log_b a$)	모두 같음	$n^d \times$ 레벨 수	$\Theta(n^d \log n)$
increasing	> 1 ($d < \log_b a$)	늘어남	마지막 항(잎) $n^{\log_b a}$	$\Theta(n^{\log_b a})$

equal 케이스의 로그 거듭제곱 일반형: 같은 분할정복 전제에서 $a/b^d = 1$ (즉, $d = \log_b a$)이고 비재귀 비용이 $f(n) = \Theta(n^d \log^k n)$ ($k \geq 0$은 고정된 상수)이면, 점화식 $T(n) = aT(n/b) + f(n)$의 해는 $T(n) = \Theta(n^d \log^{k+1} n)$이다. 이유는 깊이 i의 각 내부 레벨 비용이 $\Theta(n^d \log^k(n/b^i))$이고, 모든 내부 레벨의 비용을 합하면 $\Theta(n^d \cdot \sum_{j=1}^{\Theta(\log n)} j^k) = \Theta(n^d \log^{k+1} n)$이 되기 때문이다.

즉, 이 일반형에서는 비재귀 비용의 로그 지수가 k일 때 전체 실행 시간의 로그 지수는 $k+1$이다.

- 외울 것 없음 → a, b, d만 읽어서 $\log_b a$와 d 비교. 증명 = 레벨별 합, 등비수열 계산
- binary search: $a = 1, b = 2, d = 0$ → 공비 1 (equal) → $O(\log n)$. mergesort: $a = 2, b = 2, d = 1$ → 공비 1 → $O(n \log n)$
- b가 정수가 아니어도 됨 ($b = 4/3$ 등)
- 크기가 서로 다른 부분문제, $T(n-1)$ 꼴은 **master 불가** → 트리나 펼치기

increasing 케이스 계산: 마지막 레벨($k = \log_b n$) 비용 $a^{\log_b n} = n^{\log_b a}$. 공비 > 1이면 합은 마지막 항의 상수배.

손계산 7 · 레벨별 비용 ($n = 8$의 비용을 8^d로 표기)

레벨 k의 비용 = $n^d(a/b^d)^k$, 레벨 0-3:

점화식	k=0	k=1	k=2	k=3	합
binary search (a=1, b=2, d=0)	1	1	1	1	4
mergesort (a=2, b=2, d=1)	8	8	8	8	32
$T(n) = 2T(n/2) + n^2$ (a=2, b=2, d=2)	64	32	16	8	120
Karatsuba (a=3, b=2, d=1)	8	12	18	27	65

- 셋째 행(decreasing): 루트 64가 가장 크고 합 120 < 2 × 64
- equal: 레벨마다 같음 → 레벨 수만큼 곱해짐 (mergesort 8 × 4 = 32)
- increasing: 마지막 항(잎의 수 $a^{\log_b n}$ = 27)이 가장 큼. 합 ÷ 마지막 항 = 65/27 → 상수 이하

손계산 8 · 점화식 → 케이스

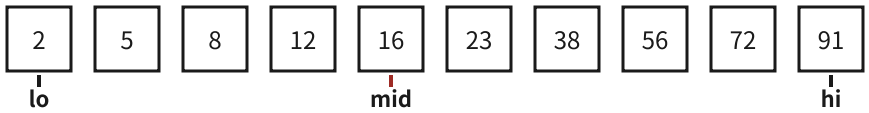
점화식	a	b	d	$\log_b a$	비교	결과
binary search	1	2	0	0	같음	$O(\log n)$
mergesort	2	2	1	1	같음	$O(n \log n)$
quicksort 최선 (중앙값 pivot)	2	2	1	1	같음	$O(n \log n)$
정수 곱셈 (4번 호출)	4	2	1	2	증가 (잎 지배)	$O(n^2)$
Karatsuba	3	2	1	1.585	증가 (잎 지배)	$O(n^{1.585})$
무작위 선택	1	4/3	1	0	감소 (루트 지배)	$O(n)$
closest pair (정렬 후)	2	2	1	1	같음	$O(n \log n)$

무작위 선택: $a = 1, b = 4/3$ (좋은 pivot이면 3n/4 이하로 줌).

Master로 못 푸는 점화식 3가지

점화식	이유	푸는 방법	결과
Hanoi $T(n) = 2T(n-1) + 1$	크기가 나뉘지 않고 1 줄어듦	펼치기	$2^n - 1$
quicksort 1/3 분할 $T(n) = T(n/3) + T(2n/3) + n$	부분문제 크기가 둘	레벨 합 $\leq n$, 깊이 $O(\log n)$	$O(n \log n)$
median of medians $T(n) = T(n/5) + T(7n/10) + n$	부분문제 크기가 둘	펼치기 (합 9/10배)	$O(n)$

이진 탐색 (Binary Search)



정렬된 배열에서 **23** 찾기. 매번 가운데(mid)와 비교해 탐색 구간을 반으로 줄인다.

◀ 이전 1 / 4 다음 ▶

정수 곱셈과 Karatsuba

문제	n비트 두 수 x, y의 곱 (n이 크면 정수형을 넘는 큰 수)
배경	덧셈·뺄셈은 비트 연산으로 $O(n)$ → 비싼 건 큰 수끼리의 곱셈
목표	**곱셈 횟수 줄이기**

복소수 곱에서 힌트

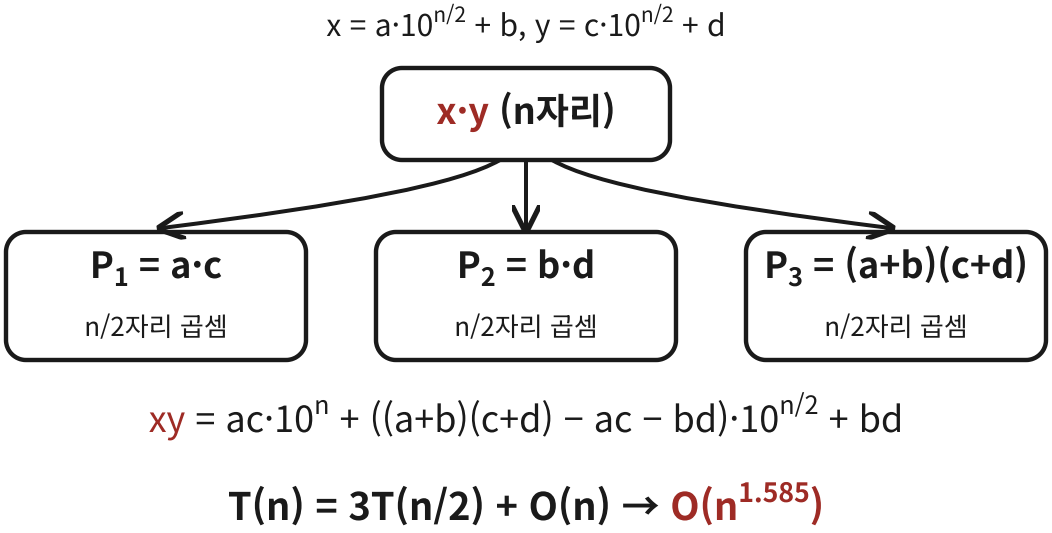
$$(a+bi)(c+di) = ac - bd + (bc+ad)i \;\Rightarrow\; \text{곱 셈 } 4\text{번} \;\rightarrow 3\text{번 : } ac,\ bd,\ (a+b)(c+d)$$

- $(a+b)(c+d) = ac + bd + bc + ad$ → ac와 bd가 식에 두 번 나옴 → 한 번만 계산
- 비트 수를 반으로 쪼갬: $x = x_L \cdot 2^{n/2} + x_R$ (n은 짝수로 가정; 홀수면 n/2 대신 ⌈n/2⌉),
  $y = y_L \cdot 2^{n/2} + y_R$ → 비트 곱이 4번 → 재귀 호출 4번, 쉬프트·덧셈은 $O(n)$

Karatsuba

$$P_1 = x_L y_L, P_2 = x_R y_R, P_3 = (x_L + x_R)(y_L + y_R)$$

$$xy = P_1 \cdot 2^n + (P_3 - P_1 - P_2) \cdot 2^{n/2} + P_2$$



- 1960년 Karatsuba
- 이미 구한 곱을 재사용 → 새 곱은 P_3 하나. $P_3 - P_1 - P_2 = x_L y_R + x_R y_L$
- 모든 재귀 레벨에서 재귀 곱셈이 4개 → 3개로 줄어, 복잡도가 $\Theta(n^2)$에서 $\Theta(n^{\log_2 3})$으로 낮아지는 점근적 성장률 개선이다 (상수배 개선이 아님)
- master: $a = 3, b = 2, d = 1 \rightarrow \log_2 3 \approx 1.585 > d \rightarrow$ increasing → $O(n^{1.585})$. 재귀 트리: 레벨 k에 부분문제 3^k개, 크기 $n/2^k$ → 레벨 비용 $n(3/2)^k$ → 등비합의 마지막 항 $3^{\log_2 n} = n^{\log_2 3}$. $O(n^2)$와 비교하면 훨씬 작음
- 같은 발상의 계보 — 곱셈이 덧셈보다 비싸니 곱 횟수를 줄임:
  - Toom-Cook (k 고정): k조각으로 쪼개 재귀 곱셈 $2k-1$번.
    점화식은 T(n) = (2k−1)·T(n/k) + O(n)이고, 지수는 $\log_k(2k-1)$이다. log(2k−1)/log k는 이와 같은 값이라 어느 한쪽이 더 작지 않다.
    $k = 2$가 Karatsuba ($\log_2 3 = 1.585$), $k = 3$이면 $\log_3 5 = 1.465$
  - 더 밀고 나가면 FFT 기반 — 1971년 Schönhage–Strassen $O(n \log n \log\log n)$: FFT 기반 첫 알고리즘, 아주 큰 n에서 빠른 방법 중 하나(실무에서는 입력 크기에 따라 다른 방법이 함께 쓰임).
    2019년 Harvey–van der Hoeven $O(n \log n)$

손계산 9 · 10110010 × 01100011 (178 × 99)

$n = 8, x_L = 1011, x_R = 0010, y_L = 0110, y_R = 0011, n/2 = 4$. 슬라이드의 세 하위 곱: 1011 × 0110, 0010 × 0011, 1101 × 1001 (= 13 × 9)

- $P_1 = x_L y_L = 11 \times 6 = 66, P_2 = x_R y_R = 2 \times 3 = 6,$
  $P_3 = (x_L + x_R)(y_L + y_R) = 13 \times 9 = 117$
- $P_3 - P_1 - P_2 = 117 - 66 - 6 = 45$
- 결과: $66 \times 2^8 + 45 \times 2^4 + 6 = 16896 + 720 + 6 = 17622$ = 178 × 99 (일치)
- 가운데 항 45 = $33 + 12$, 곱 4번 방식과 같은 값

1비트 곱 횟수 (n, 1비트까지 쪼갤 때):

n	4번 곱 방식 n^2	Karatsuba $3^{\log_2 n}$
1	1	1
2	4	3
4	16	9
8	64	27
16	256	81
1024	1048576	59049

- $n = 1024$에서 약 17.8배 차이
- $P_1 = 1011 \times 0110$도 같은 방법: 4비트라 곱 2, 6, 15 → 가운데 15 − 2 − 6 = 7, 결과
  $2 \times 16 + 7 \times 4 + 6 = 66$. $x_L + x_R$은 $n/2 + 1$비트가 될 수 있음 (1111 + 1111 = 11110) → 점근 차수는 그대로

보충 · Strassen 행렬 곱셈

고전적 행렬 곱셈

두 n × n 행렬을 각각 네 개의 크기 (n/2) × (n/2) 블록으로 나누어 곱셈을 수행할 수 있다. 이때 고전적인 블록 행렬 곱셈은 크기 n/2인 행렬 곱셈 8회와 $O(n^2)$개의 덧셈 연산을 사용한다.

따라서 시간 복잡도는 점화식 $T(n) = 8T(n/2) + O(n^2)$로 표현되고, 마스터 정리를 적용하면 $T(n) = O(n^3)$을 얻는다.

행렬의 블록 분할

Strassen의 알고리즘은 Volker Strassen이 제안한 행렬 곱셈 방법이다. 두 $n \times n$ 행렬 X, Y를 각각 네 개의 크기 $(n/2) \times (n/2)$ 블록으로 나눈다.

```
X = [ A   B ]        Y = [ E   F ]
    [ C   D ]            [ G   H ]
```

고전적인 방법에서는 이 블록들을 곱하기 위해 8회의 행렬 곱셈이 필요하다. Strassen의 알고리즘은 중간 행렬 P1, …, P7을 정의하여 행렬 곱셈 횟수를 7회로 줄인다.

일곱 개의 중간 행렬 정의

각 Pi는 크기 n/2인 행렬들의 곱으로 정의한다.

- $P1 = A(F - H)$
- $P2 = (A + B)H$
- $P3 = (C + D)E$
- $P4 = D(G - E)$
- $P5 = (A + D)(E + H)$
- $P6 = (B - D)(G + H)$
- $P7 = (A - C)(E + F)$

이와 같이 일곱 개의 중간 행렬을 계산하면, 이 결과들을 덧셈과 뺄셈으로 결합하여 최종 행렬 XY를 구할 수 있다.

중간 행렬의 결합

최종 결과 XY의 네 블록은 다음과 같이 계산한다.

- 왼쪽 위 블록 = $P5 + P4 - P2 + P6$
- 오른쪽 위 블록 = $P1 + P2$
- 왼쪽 아래 블록 = $P3 + P4$
- 오른쪽 아래 블록 = $P1 + P5 - P3 - P7$

슬라이드에는 수치 행렬을 대입한 손계산 예제가 제시되어 있지 않으므로, 여기서는 일곱 개의 곱을 정의하고 이를 결합하는 방법을 정리한다.

점화식과 시간 복잡도

Strassen의 알고리즘은 크기 n/2인 행렬 곱셈 7회와 18회의 $O(n^2)$ 행렬 덧셈·뺄셈을 수행한다.

따라서 시간 복잡도는 점화식 $T(n) = 7T(n/2) + O(n^2)$로 표현된다.

마스터 정리를 적용하면 a = 7, b = 2이고, 재귀적인 곱셈 항에 해당하는 지수는 $\log_2 7$이다. n^2 항보다 $n^{\log_2 7}$ 항이 더 빠르게 증가하므로 $T(n) = O(n^{\log_2 7}) \approx O(n^{2.81})$을 얻는다.

고전적 방법과 Strassen의 비교

구분	고전적 블록 행렬 곱셈	Strassen 알고리즘
크기 n/2 행렬 곱셈 횟수	8회	7회
덧셈·뺄셈 비용	$O(n^2)$	18회의 $O(n^2)$ 연산
점화식	$T(n) = 8T(n/2) + O(n^2)$	$T(n) = 7T(n/2) + O(n^2)$
시간 복잡도	$O(n^3)$	$O(n^{\log_2 7}) \approx O(n^{2.81})$

Strassen의 핵심은 행렬 덧셈·뺄셈 연산을 추가로 사용하여 크기 n/2인 행렬 곱셈 횟수를 8회에서 7회로 줄이는 것이다. 이 차이가 점화식의 재귀항에 반영되며, 마스터 정리를 통해 고전적 방법보다 낮은 점근적 시간 복잡도를 얻는다.

선형 시간 선택 (selection)

문제	정렬 안 된 n개 리스트 S에서 k**번째로 작은 원소** 찾기
목표	$O(n)$ (정렬하면 $O(n \log n)$)
예	60명 중 11번째로 키 큰 학생, 중앙값 = $k = n/2$

Selection의 reduction

- pivot v를 고르고 3개로 나눈 뒤 **한쪽만** 재귀: S_L (보다 작음) / S_v (같음) / S_R (큼)
- 같은 값이 여러 개 있을 수 있음 (multiset) → 같음 구역이 따로 필요. 한 번 훑으며 구역 크기도 셈 → 어느 쪽에 있는지 바로 앎
- 셋째 경우 k를 줄이는 이유: 앞 두 구역을 버렸으니 S_R 안의 순위로 바꿈. 예: $k = 10$이고 버린 원소가 5개 → S_R에서 5번째
- 최악: 매번 최솟값이 pivot → $O(n^2)$. 목표: 좋은 pivot을 빠르게 고르기

selection: pivot v로 3분할, 한쪽만 재귀

pivot v = 5

SL (v보다 작음) · 3개	Sv (= v) · 2개	SR (v보다 큼) · 6개

k ≤ |SL| → SL에서 k번째 재귀
|SL| < k ≤ |SL|+|Sv| → 답은 v
그보다 크면 → SR에서 (k − |SL| − |Sv|)번째 재귀

예: k=2 → SL 2번째 = 2 · k=4 → v = 5 · k=10 → SR 5번째 = 21

손계산 11 · 슬라이드 예제

$S = \langle 2, 36, 5, 21, 8, 13, 11, 20, 5, 4, 1 \rangle$ (11개), pivot $v = 5$:

- $S_L = \langle 2, 4, 1 \rangle$ (3개), $S_v = \langle 5, 5 \rangle$ (2개), $S_R = \langle 36, 21, 8, 13, 11, 20 \rangle$ (6개)
- $k = 2$: 조건 $2 \leq 3$ → 구역 S_L, S_L에서 2번째 → 답 2
- $k = 4$: 조건 $3 < 4 \leq 5$ → 구역 S_v → 답 $v = 5$
- $k = 10$: 조건 $10 > 5$ → 구역 S_R, S_R에서 5번째 (10 − 5) → 답 21
- 정렬 결과 1 2 4 5 5 8 11 13 20 21 36. 2번째 2, 4번째 5, 10번째 21 (일치)

무작위 pivot

- pivot v를 무작위로 고름. 운 좋으면(lucky) S_L, S_R이 균형 → $T(n) = T(n/2) + O(n) = O(n)$. 운 나쁘면 → $O(n^2)$
- 균형 = 두 구역 모두 n의 3/4 이하. pivot 순위가 25~75 백분위면 좋은 pivot. 확률 약 1/2

- 좋은 pivot이 나올 때까지 다시 고르고, 그 뒤 재귀. 한 번 고를 때마다 분할·검사에 $O(n)$. 동전 던지기에서 앞면까지 평균 횟수 $1/p = 2$
- 기대 시간 $O(n)$: $T(n) = T(3n/4) + O(n)$ → master $a = 1, b = 4/3, d = 1$ → decreasing. 레벨 합 $n(3/4)^k$의 등비합
- 기대 시간의 점화식: 좋은 pivot이 나올 때까지의 기대 시도 횟수가 상수(평균 2회)이므로 E[T(n)] ≤ E[T(3n/4)] + O(n)으로 쓸 수 있고, 기대 시간은 O(n)이다. 최악 시간 Θ(n²)은 별개로 그대로 남는다.
- 최악은 여전히 $O(n^2)$, 기대 시간만 $O(n)$ (quicksort와 같은 논리). 최악이 발생할 확률은 낮지만 0은 아니다 (무작위화가 최악을 제거하지는 않음) → **실전에서는 이 방법을 씀**

① **평균 시행 횟수 2**: 앞면 확률 $p = 1/2$. 첫 번에 앞면(확률 p)이면 1번, 아니면 1번 쓰고 처음부터

② $E = 1 + (1-p)E \to pE = 1 \to E = 1/p$ → 평균 2번

③ 연속 실패 확률은 급격히 작아짐

Median of medians

최악에도 $O(n)$을 보장하는 결정적 방법. pivot = 중앙값들의 중앙값 v (이름의 유래).

5단계

① S를 5개씩 $\lceil n/5 \rceil$개 블록으로 나눔 (마지막은 모자랄 수 있음)

② 블록마다 중앙값을 구해 배열 T에 모음 (블록당 상수 시간)

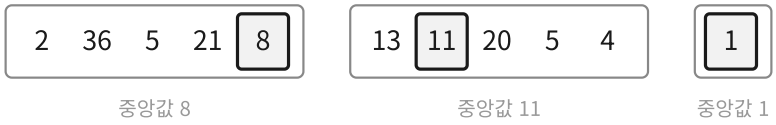
③ T의 중앙값 v을 재귀로 구함 (크기 $n/5$)

④ v으로 S를 S_L (작음), S_R (큼)로 나눔

⑤ v가 k번째면 끝, 아니면 한쪽만 재귀 (k 보정은 아래 규칙과 같음)

종료·재귀 규칙 (무작위 pivot의 selection 규칙과 동일): k ≤ $|S_L|$이면 S_L에서 k번째를 찾고, $|S_L| < k \le |S_L|+|S_V|$이면 답은 pivot v이며, k > $|S_L|+|S_V|$이면 S_R에서 k−$|S_L|$−$|S_V|$번째를 찾는다.

median of medians: 5개씩 블록 → 중앙값들의 중앙값



T = ⟨8, 11, 1⟩ → 재귀로 T의 중앙값 v = 8 (크기 n/5 재귀)

v보다 작은 중앙값 약 n/10개 × 각 블록에서 2개 더 작음 → v보다 작은 원소 ≥ 3n/10

→ 어느 쪽으로 재귀해도 크기 ≤ 7n/10 · T(n) = T(n/5) + T(7n/10) + O(n), 레벨 합 9/10배 → O(n)

- v보다 작은 중앙값 약 $n/10$개 ($\lceil n/5 \rceil / 2$). 그 중앙값마다 같은 블록 안의 2개가 더 작음 → v보다 작은 원소 $\geq 3n/10$. 대칭으로 큰 원소도 $\geq 3n/10$
- 주의: 중복 원소가 있으면 “v보다 엄격히 작은 원소 ≥ 3n/10”으로는 보장할 수 없다. 정확한 보장은 v 이하·v 이상 원소를 세는 아래 단계별 증명과 같다.
- 재귀 호출 크기 $\leq 7n/10$

크기 보장 — 단계별 증명

① 가정: 블록 크기 5, v는 블록 중앙값들의 중앙값. 중복 원소를 허용한다.

② 핵심 관찰: 블록 중앙값 중 v 이하인 것이 약 절반(≥ n/10개)이고, 그 중앙값마다 같은 블록 안에 그보다 크지 않은 원소가 2개 더 있다.

③ 수식: v 이하인 원소 ≥ 3 × (n/10) − O(1) = 3n/10 − O(1)개 (끝 블록과 중앙값 자신의 상수 개 오차 제외). 대칭으로 v 이상인 원소도 ≥ 3n/10 − O(1)개.

④ 결론: 어느 쪽으로 재귀해도 크기는 ≤ 7n/10 + O(1). 점화식 T(n) ≤ T(n/5) + T(7n/10 + O(1)) + O(n)을 펼치면 O(n).

점화식 펼치기

부분문제 크기가 둘($n/5$, $7n/10$) → master 형태 아님 → 펼침. 크기 합 $n/5 + 7n/10 = 9n/10$ → 레벨마다 9/10배:

레벨	부분문제 크기	합 (n 대비)
0	n	1

1	n/5 + 7n/10	9/10
2	n/25 + 7n/50 + 7n/50 + 49n/100	81/100
3	⋯	729/1000

- $T(n) = T(n/5) + T(7n/10) + O(n)$, 레벨 합 $n(9/10)^k$ → 합 $O(n)$
- 블록 크기가 5인 이유: **5가 지수적 감소를 주는 가장 작은 홀수**

블록 크기	재귀 크기	합	결과
3	n/3 + 2n/3	1	$n \log n$
5	n/5 + 7n/10	9/10	n
7	n/7 + 5n/7	6/7	n

- 3이면 레벨 합이 n 그대로 → quicksort 1/3 분할과 같은 $O(n \log n)$

손계산 12 · 슬라이드 예제의 median of medians

같은 S (11개), 5개씩 블록: [2, 36, 5, 21, 8] → 중앙값 8, [13, 11, 20, 5, 4] → 중앙값 11, [1] (마지막, 모자람) → 중앙값 1.

- $T = \langle 8, 11, 1 \rangle \to v = 8$
- $S_L = \langle 2, 5, 5, 4, 1 \rangle$ (5개, 8보다 작음), $S_R = \langle 36, 21, 13, 11, 20 \rangle$ (5개, 8보다 큼)
- $k = 9$: $9 > 5 + 1$ → S_R에서 $9 - 6 = 3$번째 → 20 (정렬한 11개 중 9번째와 일치)
- 크기 보장: v 이하 원소 $\geq 3n/10$, 이상 $\geq 3n/10$ → 재귀 크기 $\leq 7n/10$

무작위 pivot vs median of medians

항목	무작위 pivot	median of medians
시간	기대 $O(n)$	최악 $O(n)$
최악	$O(n^2)$	$O(n)$

점화식	$T(n) = T(3n/4) + O(n)$	$T(n) = T(n/5) + T(7n/10) + O(n)$
푸는 법	master ($b = 4/3$)	펼치기
상수	작음	큼
실전	이쪽을 씀	이론적 보장용

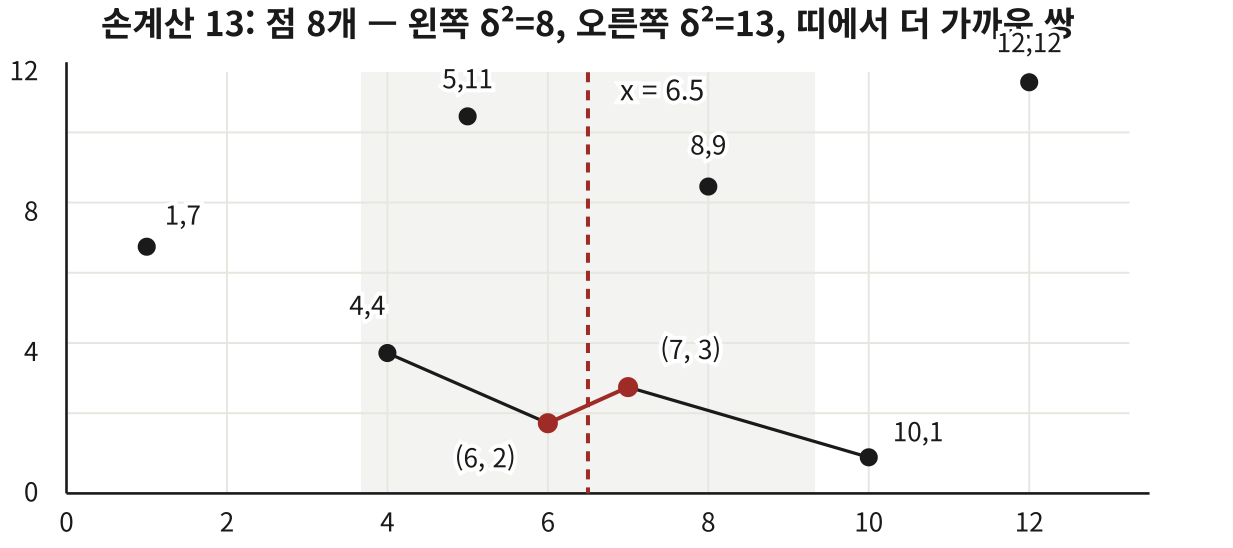
Closest pair of points

문제: 평면 위 n개 점 중 유클리드 거리가 가장 짧은 쌍. brute force: 모든 쌍 $\Theta(n^2)$ 비교.

가정: 모든 점의 x 좌표는 서로 다르다고 둔다. x 중앙값으로 나눈 두 반의 크기가 균형 잡혀야 아래 O(n log n) 분석이 성립한다. x 좌표가 같은 점이 있으면, 동점인 점들을 y 좌표 순으로 한쪽에 배정하는 동률 규칙을 먼저 정해야 한다.

분할정복과 전처리

- 중앙값 x_{mid}로 P^L, P^R 두 반으로 나눠 각각 재귀. 각 반의 최소 거리 중 작은 쪽 δ
- 빠진 경우: **한 점은 왼쪽, 한 점은 오른쪽인 쌍** → 가운데 선에서 δ 이내인 **띠(strip)**만 검사. 선에서 δ 넘게 떨어진 점은 반대쪽 점과 δ보다 가까울 수 없음
- 전처리: x 정렬 P_x, y 정렬 P_y를 **처음에 한 번만** ($O(n \log n)$). 재귀에서 P_x^L, P_x^R: P_x를 반으로 자름 ($O(1)$). P_y^L, P_y^R: P_y를 한 번 훑어 $O(n)$에 구성. **재귀마다 다시 정렬하지 않음**

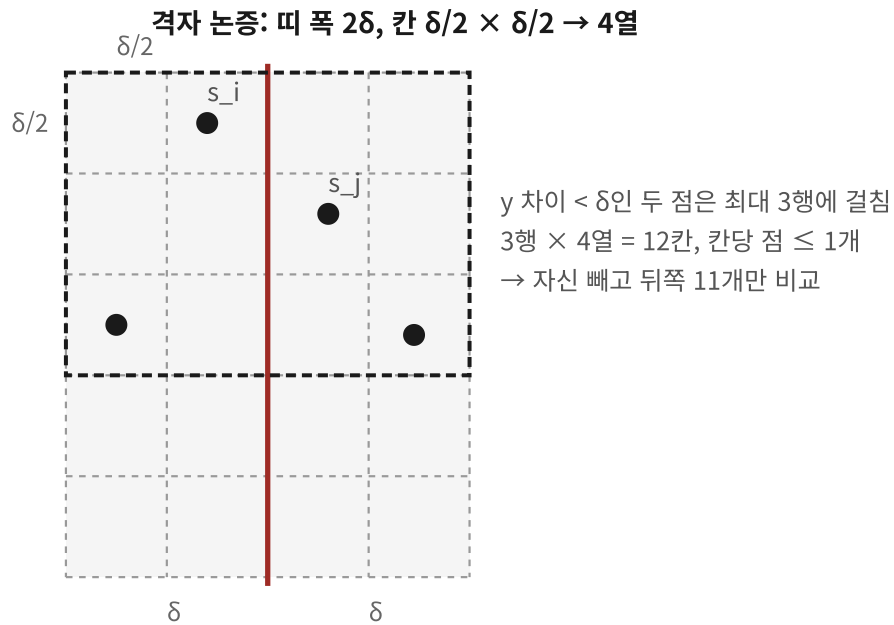


$\delta^2_L = 8$ $\delta^2_R = 13$ 정답 (6,2)–(7,3): 거리² 2

띠 $|x - 6.5| < \delta = \sqrt{8} \approx 2.83$
→ x = 4, 5, 6, 7, 8 인 점 5개만 검사

띠 검사와 격자 논증

- 띠 안의 점을 y 순서로 $s_1, s_2, \ldots$: P_y를 훑으며 띠 안의 점만 남김, $O(n)$
- 점마다 몇 개만 비교하는 이유 (격자 논증):



- $\delta/2 \times \delta/2$ 격자: 한 칸에 두 점 불가 (칸은 분할선을 넘지 않아 같은 쪽 점만 들어오고, 같은 쪽 점끼리는 δ 이상인데 칸의 대각선은 $\delta/\sqrt{2} < \delta$)
- 행 번호 차이가 3 이상인 두 점은 y 거리가 δ 이상
- **Claim**: $|i-j| \geq 12$이면 s_i와 s_j는 δ 이상 떨어짐 → 점마다 뒤의 **11개만** 비교 → 띠 안 비교 $O(n)$

$$T(n) = 2T(n/2) + O(n) \;\Rightarrow\; O(n \log n) \quad (\text{전처리 정렬도 } O(n \log n) \text{ 한 번})$$

1. **12가 나오는 이유**: 띠 폭 2δ → 칸 변 $\delta/2$로 4열
2. y 차이가 δ 미만인 두 점은 최대 3행에 걸침 (4행이면 사이에 통째로 낀 2행 때문에 y 차이 δ 이상)
3. 3행 × 4열 = 12칸, 칸당 점 1개 이하 → y 창 안에 점이 12개 이하
4. 그래서 s_{i+11} 뒤($|i-j| \geq 12$)는 항상 δ 이상 → 11개만 확인

띠 검사가 O(n)인 이유 — 단계별 정리

① 가정: 각 반 안에서는 점끼리 δ 이상 떨어져 있다 (재귀가 보장). 반대쪽 점과 δ보다 가까운 쌍만 새로 찾으면 된다.

② 핵심 관찰: 그런 쌍의 두 점은 모두 분할선에서 δ 이내, 즉 폭 2δ인 띠 안에 있어야 한다.

③ 격자 구성: 띠를 한 변 δ/2인 칸으로 나눈다. 한 칸의 대각선은 δ/√2 < δ라, 같은 칸에 같은 쪽 점이 둘 있을 수 없다.

④ 비교 개수 상한: y 차이가 δ 미만인 점들은 최대 3행 × 4열 = 12칸에 걸치고 칸당 점은 1개 이하 → 점마다 뒤쪽 최대 11개만 비교하면 된다.

⑤ 결론: 띠 검사는 O(n), 점화식 T(n) = 2T(n/2) + O(n) → O(n log n).

손계산 13 · 점 8개

$P = \{(1,7),(4,4),(5,11),(6,2),(7,3),(8,9),(10,1),(12,12)\}$, $x_{\text{mid}} = 6.5$.

- x 정렬 후 반으로: 왼쪽 4개 (x ≤ 6), 오른쪽 4개 (x ≥ 7)
- 왼쪽: 최소쌍 (4,4)-(6,2), 거리² 8. 오른쪽: 최소쌍 (7,3)-(10,1), 거리² 13. $\delta^2 = \min(8, 13) = 8$
- 띠: $|x - 6.5| < \delta = \sqrt{8} \approx 2.83$ → x = 4, 5, 6, 7, 8인 점 5개. y 순서: (6,2) (7,3) (4,4) (8,9) (5,11)
- 이웃 비교 (y 차이가 δ 이상이면 건너뜀) → 10쌍 중 4쌍만 계산:
  - (6,2)-(7,3): 거리² 2 → δ^2 **8 → 2**
  - (6,2)-(4,4): 거리² 8 → 유지. (7,3)-(4,4): 거리² 10 → 유지. (8,9)-(5,11): 거리² 13 → 유지
- 답: (6,2)-(7,3), 거리 $\sqrt{2}$. 왼쪽·오른쪽 안의 최소(8, 13)보다 작아서 가운데를 가로지르는 쌍이 정답. 띠 검사를 빼면 이 쌍을 놓침

연습문제

문제 1

원판 5개 Hanoi를 Hanoi(5, A, C, B)로 풀 때 (a) 총 이동 횟수, (b) 원판 1번과 4번의 이동 횟수, (c) 가장 큰 원판이 움직이는 순간은 몇 번째 이동인가?

힌트 보기

- $T(n) = 2T(n-1) + 1 = 2^n - 1$
- 원판 k번의 이동 횟수 = 2^{n-k}
- 가장 큰 원판은 가운데(2^{n-1}번째) 이동에서 한 번만 움직임

정답 보기

개념은 위 힌트에 정리해 둠.

1. **Step 1**: (a) $T(5) = 2^5 - 1 = 31$
2. **Step 2**: (b) 1번: $2^{5-1} = 16$회, 4번: $2^{5-4} = 2$회. 확인: 16 + 8 + 4 + 2 + 1 = 31
3. **Step 3**: (c) 앞의 4개를 B로 옮기는 데 $2^4 - 1 = 15$번 → 그 다음 이동이 5번 원판

최종 답 **(a) 31번 · (b) 1번 16회, 4번 2회 · (c) 16번째 이동**

문제 2

$L = [1, 2, 3, 5, 6, 7, 10, 13]$에서 key 10을 binary search로 찾는 과정(lo, hi, mid)을 쓰고, 정렬된 100개에서 최악 비교 횟수를 구하시오.

힌트 보기

- 가운데 $L[mid]$와 비교해 한쪽만 재귀
- $L[mid] < key$이면 lo = mid + 1, 크면 hi = mid − 1
- 최악 비교 = $\lfloor \log_2 n \rfloor + 1$

정답 보기

개념: 위 힌트 보기.

1. **Step 1**: lo 0, hi 7, mid 3, L[mid] 5, 5 < 10 → lo = 4
2. **Step 2**: lo 4, hi 7, mid 5, L[mid] 7, 7 < 10 → lo = 6
3. **Step 3**: lo 6, hi 7, mid 6, L[mid] 10, 같음 → 찾음 (3번 비교)
4. **Step 4**: 100개: $\lfloor \log_2 100 \rfloor + 1 = 6 + 1 = 7$

최종 답 **3번 비교에 찾음 · 100개 최악 7번**

문제 3

다음 점화식을 master theorem으로 풀고 케이스를 쓰시오. (a) $T(n) = 9T(n/3) + n$ (b) $T(n) = T(n/2) + n$ (c) $T(n) = 4T(n/2) + n^2$ (d) $T(n) = 3T(n/2) + n^2$ (e) 행렬을 곱 6번으로 풀 수 있다면 $T(n) = 6T(n/2) + n^2$

힌트 보기

a, b, d를 읽고 $\log_b a$와 d 비교: 작으면 increasing이면 잎이 지배 → $O(n^{\log_b a})$, 같으면 equal → $O(n^d \log n)$, 크면 decreasing이면 루트가 지배 → $O(n^d)$.

정답 보기

개념 복습은 위 힌트.

1. **Step 1**: (a) a=9, b=3, d=1 → $\log_3 9 = 2 > d$ → increasing → $O(n^2)$
2. **Step 2**: (b) a=1, b=2, d=1 → $\log_2 1 = 0 < d$ → decreasing → $O(n)$
3. **Step 3**: (c) a=4, b=2, d=2 → $\log_2 4 = 2 = d$ → equal → $O(n^2 \log n)$
4. **Step 4**: (d) a=3, b=2, d=2 → $\log_2 3 \approx 1.585 < d$ → decreasing → $O(n^2)$
5. **Step 5**: (e) a=6, b=2, d=2 → $\log_2 6 \approx 2.585 > d$ → increasing → $O(n^{2.585})$

최종 답 **(a)** $O(n^2)$ · **(b)** $O(n)$ · **(c)** $O(n^2 \log n)$ · **(d)** $O(n^2)$ · **(e)** $O(n^{2.585})$

최종 답 (Θ 표기): (a) $\Theta(n^2)$ · (b) $\Theta(n)$ · (c) $\Theta(n^2 \log n)$ · (d) $\Theta(n^2)$ · (e) $\Theta(n^{\log_2 6})$ — 세 케이스 모두 하한도 같은 차수라 O가 아닌 Θ가 정확하고, (e)는 근삿값 2.585보다 지수 표기가 정확하다.

문제 4

Karatsuba로 1101 × 1011 (13 × 11)을 계산하시오. ($n = 4$)

힌트 보기

① $x = x_L \cdot 2^{n/2} + x_R$로 쪼갠다

② $P_1 = x_L y_L, P_2 = x_R y_R, P_3 = (x_L + x_R)(y_L + y_R)$ 세 곱만 재귀

③ $xy = P_1 \cdot 2^n + (P_3 - P_1 - P_2) \cdot 2^{n/2} + P_2$

정답 보기

개념은 위 힌트 그대로.

① **Step 1**: $x_L = 11_{(3)}, x_R = 01_{(1)}, y_L = 10_{(2)}, y_R = 11_{(3)}, n/2 = 2$

② **Step 2**: $P_1 = 3 \times 2 = 6, P_2 = 1 \times 3 = 3, P_3 = (3+1)(2+3) = 4 \times 5 = 20$

③ **Step 3**: 가운데 $P_3 - P_1 - P_2 = 20 - 6 - 3 = 11$

④ **Step 4**: $6 \times 2^4 + 11 \times 2^2 + 3 = 96 + 44 + 3 = 143$

최종 답 **1101 × 1011 = 143 (= 13 × 11)**

문제 5

슬라이드의 $S = \langle 2, 36, 5, 21, 8, 13, 11, 20, 5, 4, 1 \rangle$에서 pivot $v = 8$일 때 S_L, S_v, S_R의 크기를 쓰고, $k = 7$과 $k = 3$을 selection으로 구하시오.

힌트 보기

pivot으로 3분할 후 k가 어느 구역에 있는지 크기로 판단. S_R로 가면 k를 $k - |S_L| - |S_v|$로 보정.

정답 보기

개념 정리는 위 힌트에 있음.

① **Step 1**: $S_L = \langle 2, 5, 5, 4, 1 \rangle$ (5개), $S_v = \langle 8 \rangle$ (1개), $S_R = \langle 36, 21, 13, 11, 20 \rangle$ (5개)

② **Step 2**: $k = 7$: $7 > 5 + 1 \rightarrow S_R$에서 $7 - 6 = 1$번째 → 11

③ **Step 3**: $k = 3$: $3 \leq 5 \rightarrow S_L$에서 3번째 → 4

최종 답 $k = 7$ → **11** · $k = 3$ → **4** (정렬한 11개의 7번째 11, 3번째 4와 일치)

문제 6

closest pair에서 δ일 때 격자 한 칸의 한 변과 대각선 길이, 같은 칸에 같은 쪽 점이 둘 있을 수 없는 이유, 띠 안에서 한 점이 비교할 뒤쪽 점이 11개인 이유를 쓰시오.

힌트 보기

- 띠 폭 2δ에 칸 변 $\delta/2$ 격자 → 4열
- 같은 쪽 두 점은 서로 δ 이상
- y 차이 δ 미만이면 최대 3행에 걸침

정답 보기

개념은 위 힌트에 정리해 둠.

① **Step 1**: 칸 변 $\delta/2$, 대각선 $\delta/\sqrt{2}$

② **Step 2**: 같은 쪽 두 점은 서로 δ 이상 떨어져 있음 → 한 칸에 둘이면 거리 $\leq$ 대각선 $\delta/\sqrt{2} < \delta$라 모순

③ **Step 3**: 띠 폭 2δ → 4열. y 차이가 δ 미만인 두 점은 높이 $\delta/2$짜리 행 최대 3개에 걸침 → 3행

④ **Step 4**: 4열 × 3행 = 12칸, 칸당 점 1개 이하 → 창 안에 점 12개 이하

최종 답 **칸 변 $\delta/2$, 대각선 $\delta/\sqrt{2}$ · 자기 자신 제외 뒤쪽 11개만 비교하면 충분**